Runview — Security Audit Report

Scope: Full application (FastAPI backend, React SPA, PostgreSQL, Docker deployment) on the Integration branch. **Method:** Static review of auth, RBAC, input handling, secrets, AI safety, transport, container/infra, and data protection. **Overall posture: MODERATE → GOOD.** Strong fundamentals (crypto, XXE hardening, RBAC, container hardening). Residual gaps are mostly operational/architectural and matter most if the app moves from single-org self-hosted to multi-tenant SaaS.

Executive summary

Area	Rating	One-line
Authentication	🟢 Low risk	bcrypt + HS256 JWT (24h), login rate-limiting, break-glass admin reset; optional OIDC SSO (PKCE + JWKS, link-by-email)
Authorization / RBAC	🟡 Medium	Solid role guards, but **no per-project/tenant data scoping** — every authenticated user sees all projects
Input handling / injection	🟢 Low	defusedxml (XXE), SQLAlchemy (no raw SQL), upload size + path-traversal guards, Pydantic validation
Secrets management	🟢 Low	Fernet encryption at rest for integration creds; env-based; nothing secret committed
AI / Bedrock safety	🟡 Medium	Encrypted creds, concurrency/backoff; **no explicit prompt-injection hardening**
Network / transport	🟡 Medium	No CORS or security headers configured; TLS assumed at the proxy
Container / infra	🟢 Low	Non-root, multi-stage, network isolation, dropped caps, mem/PID limits, pinned deps
Data protection	🟡 Medium	Auto-retention works; **hard-delete with no deletion audit**, no encryption-at-rest by default

---

1. Authentication — 🟢 Low

Strengths

- **bcrypt** password hashing with 72-byte truncation guard — `backend/app/auth.py`.
- **HS256 JWT**, secret required ≥16 bytes (app refuses to start otherwise), 24h expiry — `backend/app/auth.py`, `config.py`.
- **Login rate limiting** — 10 attempts / IP / 60s, sliding window — `backend/app/ratelimit.py`.
- **Admin bootstrap + break-glass** — admin seeded from env; `ADMIN_PASSWORD_RESET=1` allows reboot-time reset — `backend/app/seed.py`.

- **Client** — token in `localStorage`, cleared on logout, 401 → redirect to `/login` — `frontend/src/api.ts`.
- **SSO / OIDC (optional, opt-in)** — generic OpenID Connect (Okta/Azure/Google/Auth0). **Off by default**; a superadmin configures and enables it in **Settings → Access**, stored in `AppSetting` with the **client secret Fernet-encrypted at rest and never returned by the API** (`backend/app/sso_settings.py`; env `OIDC_*` remain a fallback). Implements authorization-code flow with **PKCE (S256)**, server-side **state + nonce**, and **ID-token validation via the provider JWKS** (signature, issuer, audience, nonce) — `backend/app/sso.py`. **Link-existing-by-email only**: SSO never auto-creates accounts (no IdP-side account injection). Config read/write is **superadmin-guarded** (`GET` / `PUT /api/settings/sso`). Roles unchanged. In SSO-only mode a superadmin password break-glass remains so an IdP outage can't cause lockout — `backend/app/sso_routes.py`, `routes.py`.

Gaps

- **LOW** — weak password policy (6-char min, no complexity) — `backend/app/schemas.py`. → Raise to 12 chars or add complexity rules.
- **LOW** — no self-service password reset; only superadmin can reset a user. → Add email-based reset or document the admin flow.
- **LOW (SSO)** — the SSO callback returns the app JWT to the SPA via URL **fragment** (standard SPA pattern; fragments aren't sent to servers/logs). Acceptable; if you prefer, switch to a one-time code exchange. Per-login **state/PKCE store is in-memory** (single-process) — a multi-replica deployment needs a shared store. Consider an `SSO_ALLOWED_DOMAINS` allow-list as defense-in-depth even though link-by-email already bounds access to provisioned users.

---

2. Authorization / RBAC — 🟡 Medium

Strengths

- Roles `viewer / editor / superadmin` enforced via `require_role()` dependency — `backend/app/auth.py`.
- Destructive/admin ops locked to superadmin (user mgmt, Bedrock/TestSigma/retention settings, TM sync/revamp/consolidate).
- Anti-lockout guards: can't demote the last superadmin; can't delete your own account — `backend/app/routes.py`.
- CI ingestion uses per-project **API keys** (bcrypt-verified, shown once) — `backend/app/auth.py`.

Gaps

- **HIGH (for multi-tenant only)** — **no per-project/workspace data scoping**: any authenticated user (any role) can read all projects' runs/results. Acceptable for a single trusted org; a data-leak risk across teams/tenants. → Add `workspace_id` to `User` and filter all queries, or run one instance per tenant, or use Postgres row-level security.
- **MEDIUM** — API keys are project-scoped but not further restricted (env/suite). → Optional `allowed_environments` / `allowed_suites` on `ApiKey`. (A leaked key is still bounded to its one project.)

3. Input handling & injection — 🟢 Low

Strengths

- **XXE** — JUnit parser uses `defusedxml`; message/trace truncated — `backend/app/junit_parser.py`.
- **SQL injection** — all access via SQLAlchemy; the only `text()` usage is hardcoded migration DDL, no user input — `backend/app/database.py`.
- **Uploads** — 25 MB cap enforced before parse; filenames sanitized via `Path(name).name` + UUID prefix (no path traversal) — `backend/app/routes.py`, `bedrock.py`.
- **Validation** — Pydantic models on all bodies; e.g. project slug regex `^[a-z0-9-]+$`.

Gaps

- **MEDIUM** — **SSRF**: context-doc URL ingestion accepts any `http(s)` URL and fetches it server-side; private ranges / `169.254.169.254` not blocked. → Block private/link-local CIDRs, disable redirects, or require an allowlist/admin approval.
- **LOW** — review the Mabl JSON parser for depth/type limits.

---

4. Secrets management — 🟢 Low

Strengths

- **Encryption at rest** — Bedrock keys, TestSigma/Jira/Rovo/GitHub tokens stored Fernet-encrypted (`enc:` prefix) in `AppSetting`; key derived from `SECRET_KEY` — `backend/app/crypto.py`.
- Env-based config; `.env` git-ignored; **no secrets committed** (verified in the deploy diff).
- `SECRET_KEY` and `POSTGRES_PASSWORD` are required at startup.

Gaps

- **LOW** — `.env.example` ships weak placeholders (`change-me-…`). → Emphasize `openssl rand -hex 32` and password-manager values in docs.
- **OPERATIONAL** — for production, prefer a secrets manager (AWS Secrets Manager / Vault) over a plaintext `.env` on the host.

---

5. AI / Bedrock safety — 🟡 Medium

Strengths

- Credentials encrypted; never returned via API (only boolean "configured" flags).
- Concurrency capped (interactive + background lanes, semaphores), exponential backoff, streaming with long read timeout; failures are caught and never crash the app — `backend/app/bedrock.py`.

- Token usage + estimated cost logged to `AiUsage`.

Gaps

- **MEDIUM** — user/code text is sent to Bedrock without explicit **prompt-injection** hardening. → Wrap user input with role-anchoring system framing, log inputs/outputs for audit, consider guardrails for sensitive flows. (Mitigated somewhat: model output is parsed into structured fields, not echoed raw to other users.)

---

6. Network & transport — 🟡 Medium

Gaps

- **LOW/MEDIUM** — **no CORS middleware**. Fine if SPA and API are same-origin (they are, bundled together); add `CORSMiddleware` with an explicit allowlist if the API is consumed cross-origin.
- **LOW** — **no security headers** (`X-Frame-Options`, `X-Content-Type-Options`, `HSTS`, `CSP`). → Add a small response-header middleware.
- **OPERATIONAL** — app does not terminate TLS; deploy behind nginx/Traefik and run uvicorn with `--proxy-headers`.

Strength — login rate limiting (single-process, in-memory; use a shared store if scaling to multiple replicas).

---

7. Container & infrastructure — 🟢 Low

Strengths

- Non-root `appuser` (UID 10001); multi-stage build (no build tools in runtime image); Python 3.12-slim / Node 22-alpine.
- Healthchecks on api + db; `no-new-privileges:true`; `cap_drop: [ALL]` with minimal adds; memory & PID limits.
- Postgres on an internal network, **not host-exposed**; API bound to `127.0.0.1:8080`.
- Python deps **pinned** to exact versions.

Gaps

- **LOW** — add **Trivy** image scan + SBOM to CI.
- **LOW** — `pgvector/pgvector:pg17` is a floating tag → pin to a digest/specific minor.

---

8. Data protection — 🟡 Medium

Strengths

- Daily **retention purge** (configurable) cascades to tests/logs/attachments and removes disk files — `backend/app/retention.py`.
- TM module has rich **audit logging** (`tm_audit_events`) and soft-delete (`archived_at`) for test-management entities.
- `AiUsage` denormalizes email/slug to preserve history after deletes.

Gaps

- **MEDIUM — core** results purge is a hard delete with **no deletion audit / no recovery**. → Add soft-delete + an `AuditLog` of who deleted what (relevant for SOC2/GDPR/HIPAA).
- **LOW — no encryption at rest** for DB/attachment volumes by default. → Enable EBS/dm-crypt (and optionally pgcrypto) in production.
- Workspace isolation is **logical** (FK + query filters) not enforced per-user (see §2).

---

9. Prioritized remediation

Priority	Item	Effort
1	Security-headers middleware (X-Frame-Options, nosniff, HSTS, CSP)	~1–2 h
2	SSRF guard on context-doc/URL fetch (block private CIDRs, no redirects)	~half day
3	Soft-delete + deletion audit log for core results	~1–2 days
4	Prompt-injection framing + Bedrock I/O audit logging	~1 day
5	Trivy scan + SBOM in CI; pin Postgres digest	~half day
6	Production hardening doc: TLS proxy, secrets manager, EBS encryption, firewall to CI/admin IPs	~1 day
7 *(multi-tenant only)*	Per-workspace data scoping (or RLS / per-tenant instances)	project

---

10. Deployment security checklist (production)

- [ ] TLS terminated at reverse proxy; uvicorn `--proxy-headers`; HSTS on.
- [ ] `SECRET_KEY` = `openssl rand -hex 32`; strong `ADMIN_PASSWORD`; rotate via secrets manager.
- [ ] Postgres + attachment volume encrypted at rest; daily backups + tested restore.
- [ ] API port reachable only from CI/CD + admin networks.
- [ ] Security headers + CORS allowlist configured.
- [ ] Trivy scan clean (HIGH/CRITICAL) before release.
- [ ] Confirm `seed_sample_data` disabled on hosted (no demo data) and migrations applied to head.